



Universidad Autónoma de Querétaro
Facultad De Ingeniería



**FACULTAD DE
INGENIERÍA**

Minimo Funcion

Algoritmos Metaheurísticos Examen 1

P R E S E N T A

Ing. Daniel Eduardo González Alvarado

Profesor

Dr. Marco Antonio Aceves Fernández

20 de Octubre de 2025

Índice

1. Introducción	1
2. Desarrollo	2
2.1. Planteamiento del problema y codificación	2
2.2. Generar población inicial	2
2.3. Selección: torneo con k participantes	2
2.4. Cruza: BLX- α (Blend Crossover)	3
2.5. Mutación gaussiana adaptativa	3
2.6. Elitismo	4
2.7. Estrategias anti-estancamiento	4
2.8. Criterios de paro	4
2.9. Complejidad y parámetros	4
2.10. Visualización y trazabilidad	5
2.11. Resumen del ciclo genético	5
2.12. Implementacion	5
2.12.1. minimo	5
2.12.2. Maximo	8
3. Resultados	10
4. Discusión	11
5. Conclusión	11

1. Introducción

La función de Rosenbrock, propuesta originalmente por Howard H. Rosenbrock en 1960, es un referente clásico dentro del ámbito de la optimización matemática y computacional. Su relevancia viene en la complejidad de su paisaje de búsqueda: una superficie continua, suave y no convexa, caracterizada por un estrecho valle curvado que conduce al mínimo global. Esta estructura la convierte en un ejercicio ideal para evaluar el desempeño y la estabilidad de algoritmos de optimización, especialmente aquellos inspirados en procesos evolutivos o heurísticos como es nuestro caso.

En este caso abordaremos la función de Rosenbrock con el fin de obtener su mínimo y máximo, además de analizar la capacidad de los métodos para escapar de regiones de estancamiento y converger hacia soluciones óptimas en espacios con topologías complejas.

El enfoque adoptado se centra en una implementación capaz de aproximar el mínimo global de la función mediante estrategias adaptativas de selección y mutación. Esta aproximación nos beneficia debido a su naturaleza estocástica, que permite un equilibrio entre exploración y explotación del espacio de búsqueda, fomentando la diversidad poblacional. De este modo, la optimización de una función de prueba tan exigente como la de Rosenbrock constituye un escenario idóneo para examinar la robustez, convergencia y comportamiento dinámico del algoritmo propuesto. La función de Rosenbrock en dos dimensiones se define matemáticamente como:

$$f(x, y) = (a - x)^2 + b(y - x^2)^2 \quad (1)$$

donde a y b son parámetros que determinan la forma del valle, siendo comúnmente $a = 1$ y $b = 100$. El mínimo global de esta función se encuentra en el punto $(x, y) = (a, a^2)$.

En esta ocasión usaremos la función acotada en el rango $[-5, 10]$ para ambas variables x y y , por lo que nuestra función queda de la siguiente manera:

Función de Rosenbrock (3D)

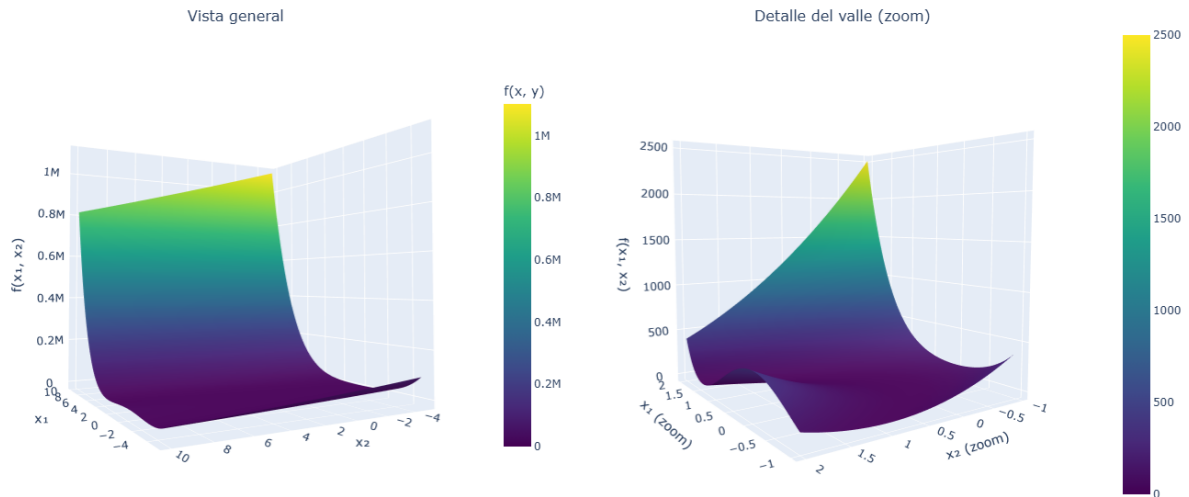


Figura 1: Representación gráfica de la función de Rosenbrock en 3D, con zoom en el valle.

2. Desarrollo

La Funcion de Rosenbrock nos presenta una curva similar a un medio circulo, con lo que encontrar el minimo es posible hasta cierto punto, buscarlo de manera visual, es decir podemos determinar una zona minima. Sin embargo como podemos observar el resultado final el minimo se encuentra ligeramente desplazado de donde se esperaria. Esto se debe a la misma funcion que cuenta con una elevacion en la parte inferior como se puede observar en el zoom de la Figura 1.

2.1. Planteamiento del problema y codificación

Se busca aproximar el mínimo global de la función de Rosenbrock

$$f(x, y) = 100(y - x^2)^2 + (x - 1)^2,$$

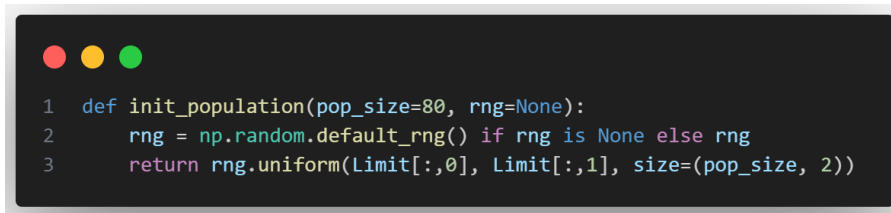
en un dominio acotado $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$. Codificaremos a nuestros individuos, de tal manera que cada solución es un vector $\mathbf{z} = (x, y) \in \mathbb{R}^2$. Esta codificación evita los costos de decodificación de los esquemas binarios y nos permite aplicar operadores continuos (cruza y mutación).

2.2. Generar población inicial

Se genera una población inicial $\mathcal{P}_0 = \{\mathbf{z}^{(i)}\}_{i=1}^N$ muestreando uniformemente cada dimensión dentro de los límites:

$$x^{(i)} \sim \mathcal{U}(x_{\min}, x_{\max}), \quad y^{(i)} \sim \mathcal{U}(y_{\min}, y_{\max}).$$

Este muestreo provee una buena diversidad inicial y cobertura del espacio de búsqueda sin sesgos hacia regiones específicas del valle. Al seleccionar una semilla aseguramos la reproducibilidad del experimento.



```
1 def init_population(pop_size=80, rng=None):
2     rng = np.random.default_rng() if rng is None else rng
3     return rng.uniform(Limit[:,0], Limit[:,1], size=(pop_size, 2))
```

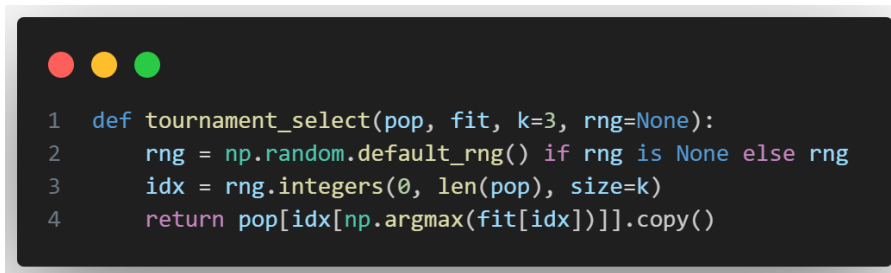
Figura 2: Funcion Generadora de Poblacion Inicial

2.3. Selección: torneo con k participantes

Usaremos **selección por torneo** de tamaño k . Para cada padre, se sortean k individuos al azar y se elige el de mejor aptitud. De tal manera que:

1. Se controla la presión selectiva por medio de k : valores mayores aceleran la explotación; valores menores preservan diversidad.
2. Es robusta ante reescalamientos de aptitud y evita dependencias de rangos absolutos.

Al implementarlo, la selección se invoca N veces para formar un multiconjunto de padres $\mathcal{M}$ con igual tamaño que la población.



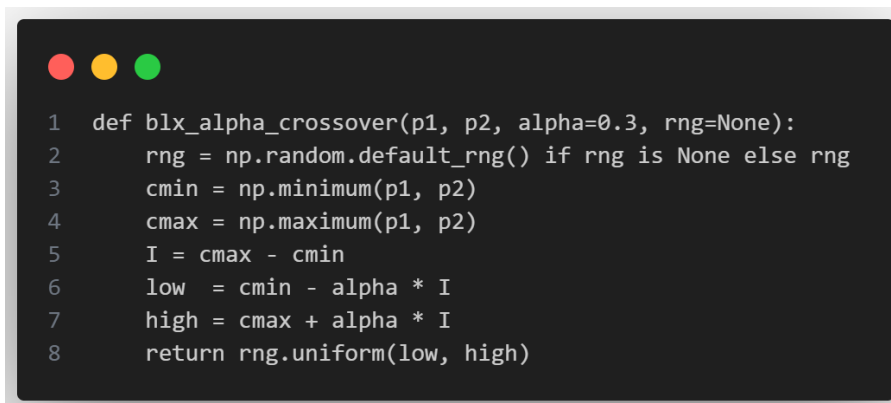
```
1 def tournament_select(pop, fit, k=3, rng=None):
2     rng = np.random.default_rng() if rng is None else rng
3     idx = rng.integers(0, len(pop), size=k)
4     return pop[idx[np.argmax(fit[idx])]].copy()
```

Figura 3: Funcion de Seleccion por Torneo

2.4. Cruza: BLX- α (Blend Crossover)

Para padres $\mathbf{p}_1$ y $\mathbf{p}_2$, el operador BLX- α crea descendientes muestreando, en cada gen j , dentro del intervalo extendido

Se seleccionó BLX- α es debido a dos razones: (i) extrapolación controlada para cruzar valles curvos sin encasillarse en el segmento entre los padres, e (ii) parametrización simple con α que equilibra exploración y explotación.

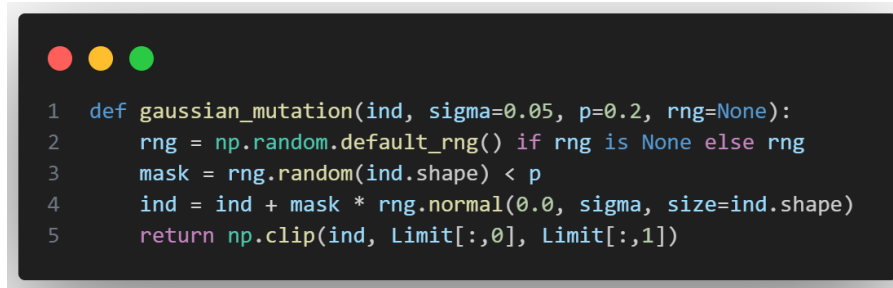


```
1 def blx_alpha_crossover(p1, p2, alpha=0.3, rng=None):
2     rng = np.random.default_rng() if rng is None else rng
3     cmin = np.minimum(p1, p2)
4     cmax = np.maximum(p1, p2)
5     I = cmax - cmin
6     low = cmin - alpha * I
7     high = cmax + alpha * I
8     return rng.uniform(low, high)
```

Figura 4: Funcion de Cruza BLX-alpha

2.5. Mutación gaussiana adaptativa

En cada generación, los individuos pueden modificarse ligeramente aplicando una mutación con una probabilidad p . Esta mutación consiste en agregar un pequeño ruido aleatorio ϵ tomado de una distribución normal $\mathcal{N}(0, \sigma^2)$. El valor de σ controla qué tan grandes son las variaciones y se reduce progresivamente a lo largo de las generaciones. De esta forma, al inicio el algoritmo explora con cambios amplios en las soluciones ($\sigma_{\text{máx}} \approx 0,12$) y, conforme avanza, realiza ajustes más finos ($\sigma_{\text{mín}} \approx 0,01$) cerca del mínimo. Después de aplicar la mutación, los valores que salgan del rango permitido se limitan a los bordes del dominio definido.



```

1 def gaussian_mutation(ind, sigma=0.05, p=0.2, rng=None):
2     rng = np.random.default_rng() if rng is None else rng
3     mask = rng.random(ind.shape) < p
4     ind = ind + mask * rng.normal(0.0, sigma, size=ind.shape)
5     return np.clip(ind, Limit[:,0], Limit[:,1])

```

Figura 5: Funcion de Mutacion Gaussiana Adaptativa

2.6. Elitismo

Para asegurar que las mejores soluciones no se pierdan, se conserva un pequeño porcentaje ρ de los individuos más aptos (alrededor del 5 %) sin modificaciones. Estos individuos llamados super individuos se copian directamente a la siguiente generación antes de aplicar los procesos de selección, cruce y mutación. El uso del elitismo nos ayuda a mantener la calidad de las soluciones, acelera la convergencia del algoritmo y evita que el azar elimine individuos que ya se encuentran cerca del mínimo global.

2.7. Estrategias anti-estancamiento

Para evitar el estancamiento se aplicaron dos estrategias simples que mantienen la diversidad de la población y permiten seguir avanzando hacia mejores soluciones:

1. **Ajuste adaptativo de la mutación:** si el mejor valor no mejora después de varias generaciones, se aumenta ligeramente el valor de σ o la probabilidad de mutación p . Esto introduce más variación en los individuos y ayuda a nuestro algoritmo a explorar nuevas zonas del espacio de búsqueda.
2. **Resiembra parcial:** cuando el progreso se detiene por completo, se reemplaza aleatoriamente entre un 5 % y un 10 % de la población, manteniendo siempre a los mejores individuos sin cambios. Esta acción reintroduce diversidad sin afectar el elitismo.

2.8. Criterios de paro

Se combinan dos criterios: límite de generaciones T y un criterio de no-mejora de la mejor aptitud global durante T generaciones consecutivas. Con esto el límite T limita el costo computacional y la no-mejora detiene la búsqueda cuando el progreso es nulo.

2.9. Complejidad y parámetros

En cada generación el algoritmo evalúa a N individuos y aplica los operadores de selección, cruce y mutación una vez por cada uno, por lo que el costo computacional principal crece de forma lineal con el tamaño de la población, es decir, $O(N)$. La evaluación de la función objetivo $f(x, y)$ es muy rápida ($O(1)$), por lo que el tiempo total depende principalmente del número de individuos y de generaciones.

Los valores de los parámetros se seleccionaron de manera empírica, buscando un equilibrio entre la exploración del espacio de búsqueda y la velocidad de convergencia:

- **Población** (N): De 100 a 1,000 individuos para mantener la diversidad sin aumentar demasiado el costo computacional.
- **Tamaño del torneo** ($k = 3$): proporciona una presión selectiva moderada, adecuada para la forma estrecha del valle de Rosenbrock.
- **Elitismo** ($\rho \approx 0,05$): conserva los mejores individuos sin frenar la aparición de nuevas soluciones.
- **Cruza BLX- α** ($\alpha \approx 0,3-0,35$): permite una combinación controlada entre exploración y estabilidad.
- **Mutación** ($p \approx 0,2-0,25$, σ_t decreciente): asegura una exploración más amplia al inicio y ajustes más finos en las últimas generaciones.

2.10. Visualización y trazabilidad

Para monitorear el progreso de nuestro algoritmo graficamos nuestra función cada cierto número de generaciones, esto nos permite observar la convergencia de manera clara. Podemos ver como cada punto (individuo) se va acercando al mínimo/máximo global indicado por un rombo rojo en nuestro gráfico.

2.11. Resumen del ciclo genético

1. Evaluar aptitud de $\mathcal{P}_t$ y extraer elites E_t .
2. Construir el conjunto de padres con torneo de tamaño k .
3. Generar hijos mediante BLX- α por parejas; aplicar clip.
4. Mutar con probabilidad p usando σ_t adaptativa; aplicar clip.
5. Insertar elites en la descendencia y formar $\mathcal{P}_{t+1}$.
6. Actualizar mejor valor global y aplicar, si corresponde, mecanismos anti-estancamiento.

2.12. Implementación

2.12.1. mínimo

Población Inicial: 100 Generaciones: 100 Tamaño de torneo: 3

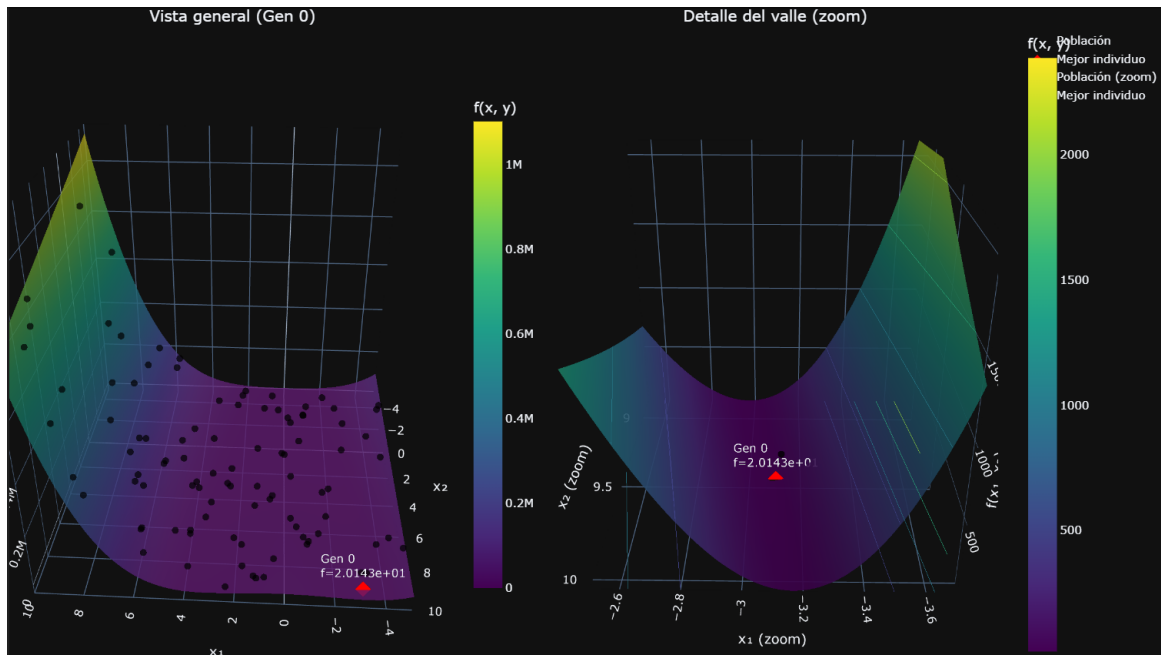


Figura 6: Funcion de Rosenbrock con poblacion inicial de 100 Gen 0

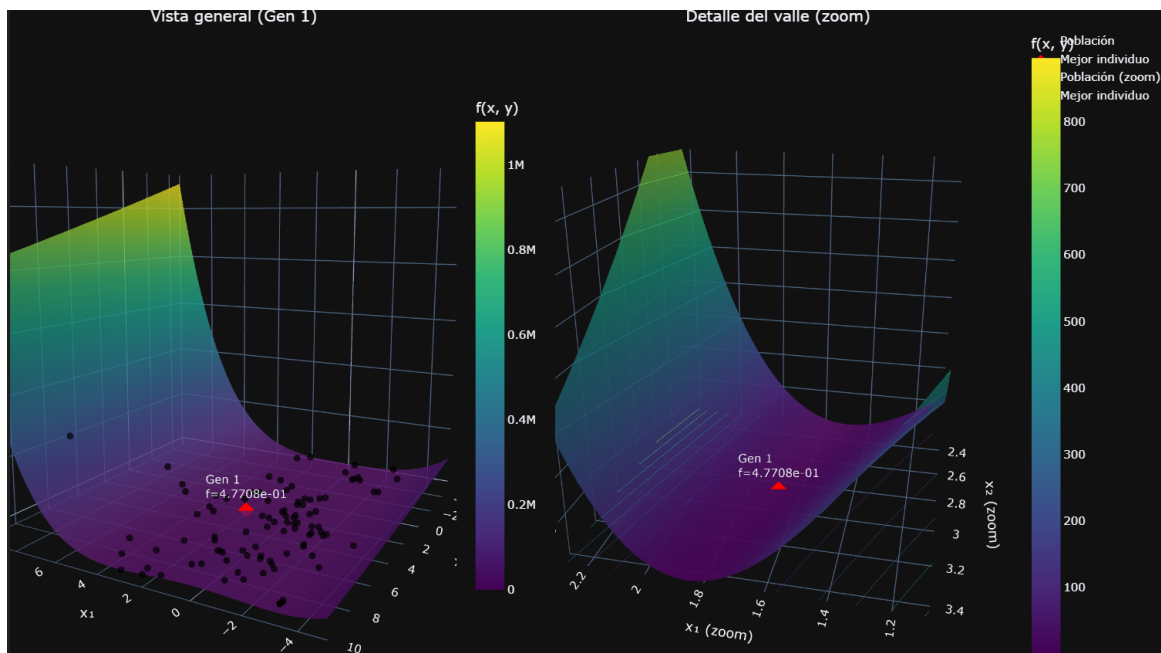


Figura 7: Funcion de Rosenbrock con poblacion inicial de 100 Gen 1

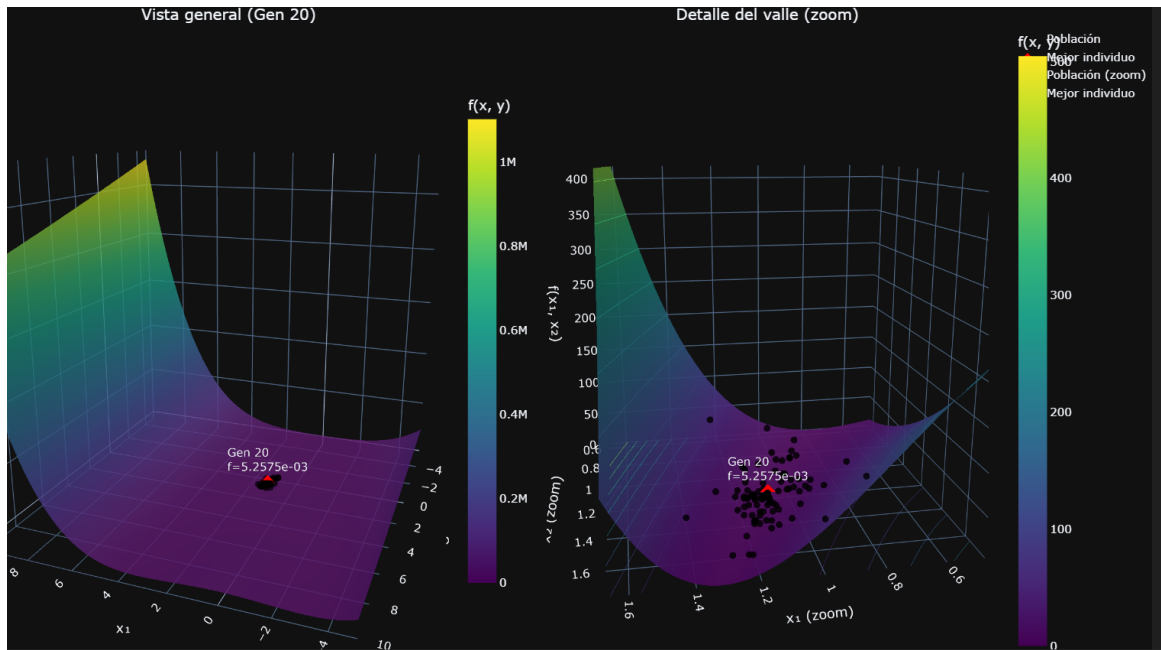


Figura 8: Funcion de Rosenbrock con poblacion inicial de 100 Gen 20

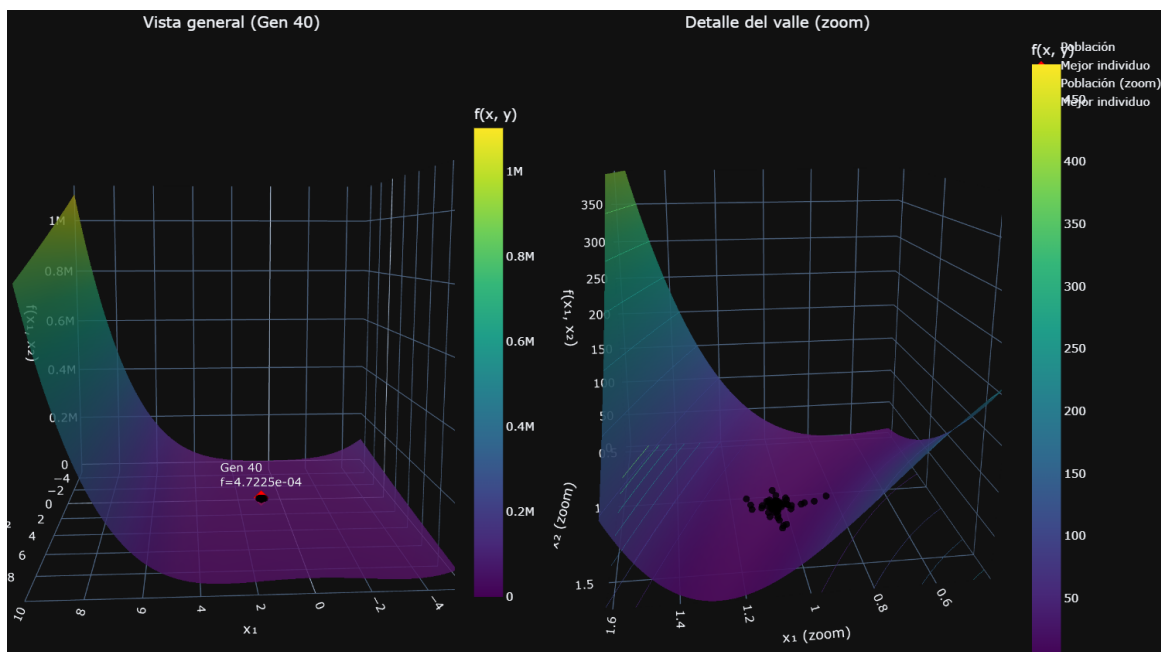


Figura 9: Funcion de Rosenbrock con poblacion inicial de 100 Gen 40

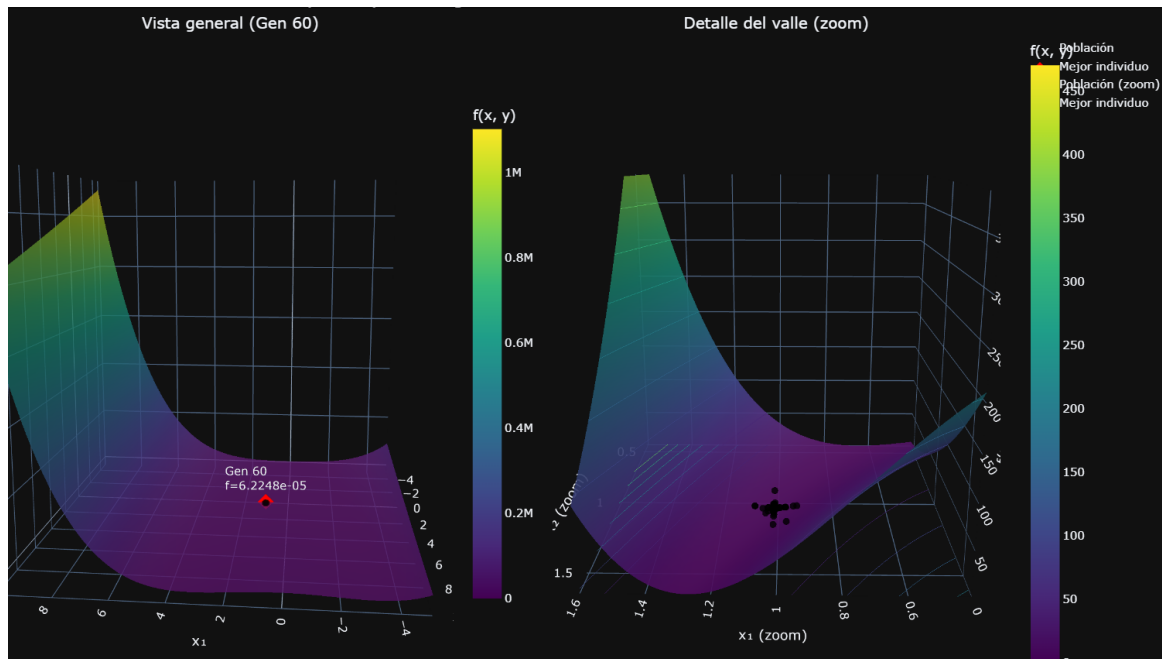


Figura 10: Funcion de Rosenbrock con poblacion inicial de 100 Gen 60

Al final nuestro algoritmo se detuvo en la generacion 68 ya que paso 10 generaciones sin cambios. Obteniendo los siguientes resultados:

- Mejor individuo: $(x=1.007874, y=1.015761)$
- Valor de la función en el mejor individuo: $6,224806e - 05$
- Generación en la que se encontró el mejor individuo: 58
- Número total de generaciones ejecutadas: 68

2.12.2. Maximo

Poblacion Inicial: 100 Generaciones: 100 Tamaño de torneo: 3

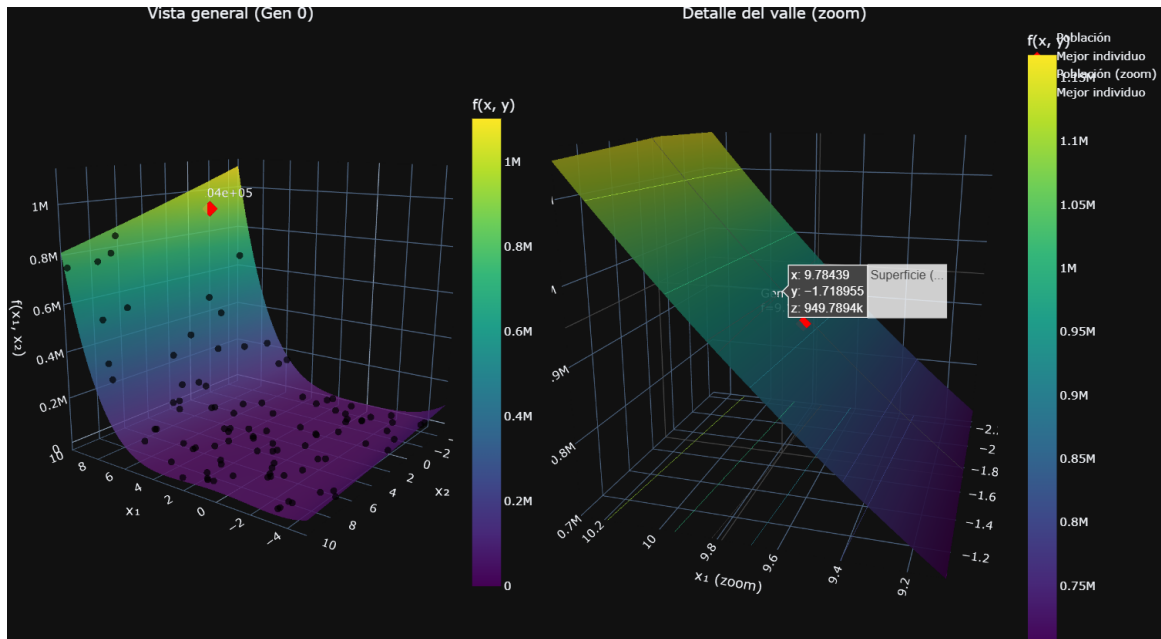


Figura 11: Funcion de Rosenbrock con poblacion inicial de 100 Gen 0

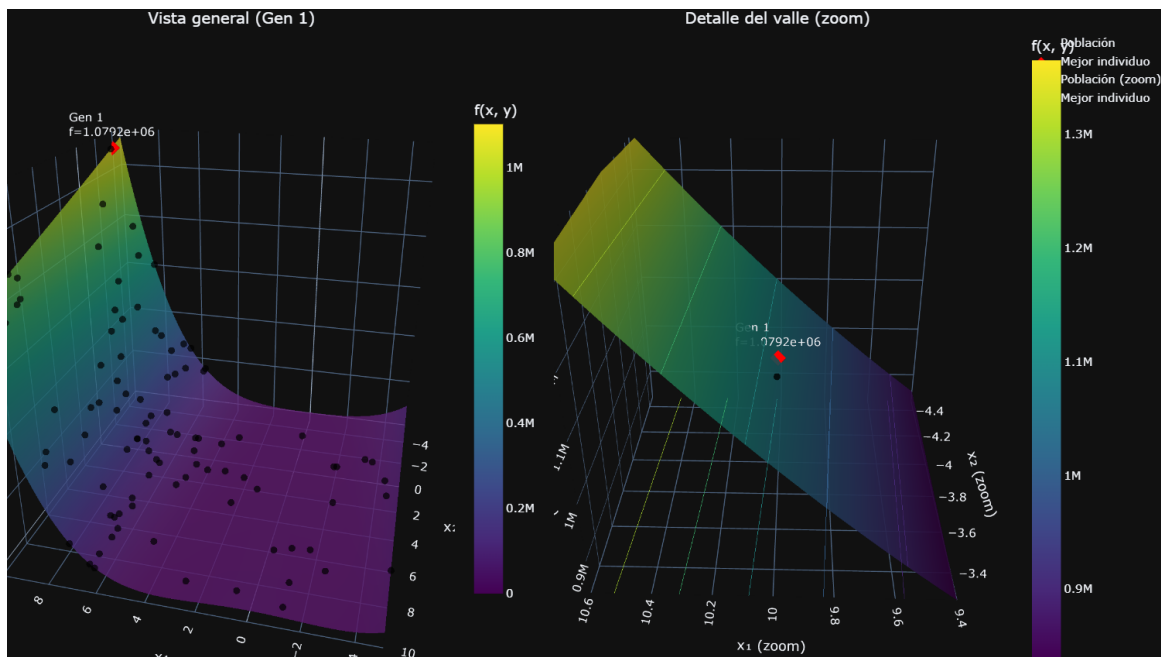


Figura 12: Funcion de Rosenbrock con poblacion inicial de 100 Gen 1

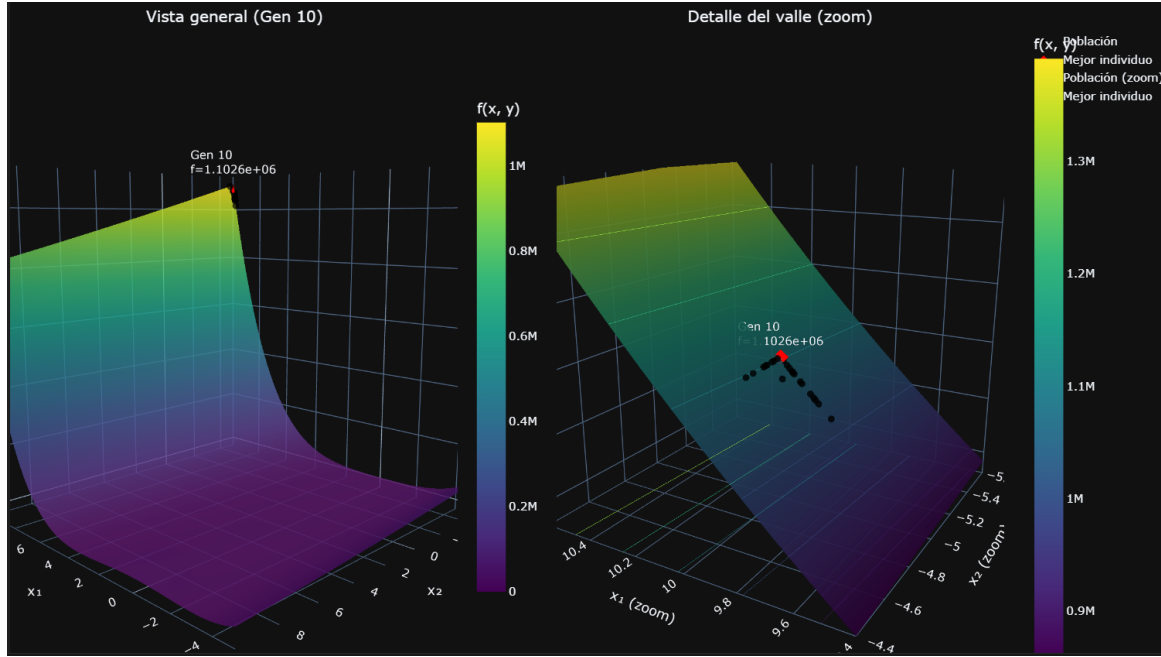


Figura 13: Funcion de Rosenbrock con poblacion inicial de 100 Gen 10

Al final nuestro algoritmo se detuvo en la generacion 12 ya que paso 10 generaciones sin cambios. Obteniendo los siguientes resultados:

- Mejor individuo: $(x=10.000000, y=-5.000000)$
- Valor de la función en el mejor individuo: $1,102581e + 06$
- Generación en la que se encontró el mejor individuo: 2
- Número total de generaciones ejecutadas: 12

3. Resultados

Como podemos observar en la figura 10 nuestro algoritmo logra encontrar el minimo global de la función de Rosenbrock en un número relativamente bajo de generaciones (alrededor de 60 generaciones), lo cual es un buen resultado considerando la complejidad de la función y el tamaño de la población (100 individuos). Además, la convergencia rápida indica que los operadores de selección, cruza y mutación están funcionando de manera efectiva para guiar a la población hacia el óptimo global.

Las gráficas nos permiten ver como la población se va acercando al mínimo global (1,1) a medida que avanzan las generaciones ademas de contar con el zoom que nos permite observar con mayor detalle el comportamiento cerca del mínimo el cual es más notorio en las ultimas generaciones.

Por otro lado en la figura 13 podemos observar que el algoritmo también es capaz de encontrar el máximo global de la función de Rosenbrock, aunque en este caso la convergencia es un poco más rápida, alcanzando el máximo en un par de generaciones (alrededor de 10 generaciones). Podemos atribuir esto a el acotamiento de nuestra función en el dominio definido, lo que facilita la búsqueda del máximo.

4. Discusión

Al generar nuestra población inicial podemos observar que los individuos están regados por todo el espacio de búsqueda, lo cual es beneficioso para la exploración inicial del algoritmo genético. De la misma manera podemos ver el mejor individuo en la generación 0, el cual podemos seguir para ver su evolución a lo largo de las generaciones. A lo largo de las generaciones podemos observar como los individuos se van acercando al mínimo de la función de Rosenbrock, lo cual es un indicativo de que el algoritmo está funcionando correctamente. Finalmente en la figura 10 podemos ver como los individuos están muy cerca del mínimo, como se ve en el zoom de la figura.

En cuanto al maximo valor de la función, podemos observar que debido a que la función esta acotada por arriba, el valor máximo no cambia mucho a lo largo de las generaciones. Y rapidamente llega a un valor cercano al máximo teórico de la función en el espacio de búsqueda definido. Incluso con una población inicial muy variada el máximo valor converge rápidamente.

En ambos casos, tanto para el mínimo como para el máximo, podemos observar que a medida que aumentamos el número de generaciones, los individuos se acercan más al valor mínimo y máximo respectivamente, lo cual es el factor determinante de la correcta implementación del algoritmo genético.

5. Conclusión

En conclusión, la implementación de algoritmos genéticos para encontrar el mínimo de la función de Rosenbrock ha demostrado ser efectiva. Durante el desarrollo, se ha demostrado una convergencia hacia el mínimo global, lo que indica que los operadores de selección, cruce y mutación están funcionando adecuadamente. Además, se ha observado que la diversidad genética en la población juega un papel crucial en la exploración del espacio de búsqueda. Los resultados obtenidos nos indican que, con un ajuste adecuado de los parámetros del algoritmo, es posible mejorar aún más la eficiencia y precisión en la búsqueda del mínimo. Esto abre la puerta a futuras investigaciones que busquen optimizar estos parámetros y explorar nuevas variantes del algoritmo genético para abordar problemas similares.

También cabe la pena mencionar que la función de Rosenbrock, es sencilla de implementar y visualizar, lo que la convierte en un excelente caso de estudio para entender el comportamiento en este caso podríamos haber explorado funciones mas complejas para ver como se comporta el algoritmo en esos casos, pero para los fines de este reporte, la función de Rosenbrock ha sido suficiente.

Por otro lado la mutación ha demostrado ser un operador crucial para mantener la diversidad en la población, evitando la convergencia prematura a óptimos locales. Sin embargo, es importante considerar que evitar la mutación en este caso en específico podría haber llevado a una convergencia más lenta pero mas sencilla, lo cual puede ser un punto a considerar en futuros experimentos.

Es decir, es importante seguir explorando y ajustando los algoritmos genéticos para usar los parametros correctos es decir cuando sean necesarios y cuando no, para maximizar su eficiencia en la búsqueda de soluciones óptimas.